

Learning Deformable Cable Geometry for Collision-Free Planning of Construction Robots

Karthik Shaji

*Daniel Guggenheim School of
Aerospace Engineering
Georgia Institute of Technology
Atlanta, USA
kshaji3@gatech.edu*

Edvard Bruun

*School of Civil and
Environmental Engineering
Georgia Institute of Technology
Atlanta, USA
edvard.bruun@ce.gatech.edu*

Frank Dellaert

*Institute for Robotics and
Intelligent Machines
Georgia Institute of Technology
Atlanta, USA
frank.dellaert@cc.gatech.edu*

Abstract—Construction robots must plan trajectories through constrained environments while accounting for collisions involving both rigid links and externally routed components. In particular, deformable cables may contact obstacles or become snagged if their configuration-dependent geometry is omitted from the collision model. We present a neural-network method for predicting quasi-static cable geometry on a 9-DoF robotic manipulation platform. The cable is represented using an endpoint-constrained Chebyshev interpolant and incorporated into a sphere-based collision-avoidance formulation. We evaluate the method in simulation on a task that requires the robot end effector and attached cable to traverse a window-like obstacle containing hooks.

Index Terms—motion planning, collision avoidance, deformable linear objects, neural networks, construction robotics

I. INTRODUCTION

Robotic systems can support construction tasks that require operation in constrained workspaces, obstacle avoidance, and precise manipulation [1]. When combined with advanced sensing and actuation mechanisms [2], these systems may reduce workers' exposure to hazardous operations and enable construction processes that require high accuracy.

Operation in such environments requires collision-free motion planning [3]. Existing approaches in construction robotics include emergency stopping and programmable-logic-controller-based replanning [3], as well as constraint-based planning using Dijkstra's algorithm [4]. Applying these approaches to high-dimensional robotic systems in constrained workspaces can be computationally challenging. We therefore consider motion-planning methods developed for general robotic systems.

Robotic manipulators often include cables routed between their joints. In construction environments, these cables may contact or snag on protruding surfaces, potentially damaging the cable or interrupting robot operation. Their geometry is challenging to model because slack, gravity, and joint motion cause configuration-dependent deformation. Collision-aware planning should therefore account for changes in cable

This material is based upon work supported by the National Science Foundation Graduate Research Fellowship Program under Grant No. DGE-2039655. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

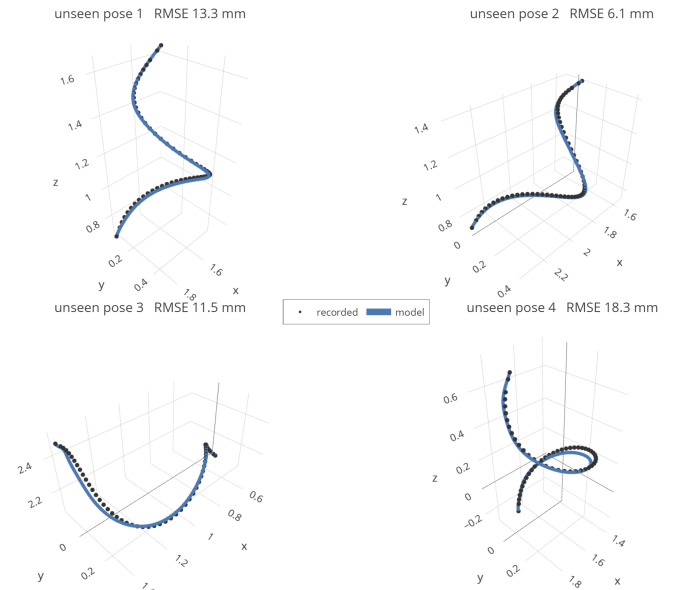


Fig. 1. Our neural network predicts the deformed shape of a robot-mounted cable directly from the robot's joint configuration. Shown are predicted cable splines (solid) against simulated ground truth (dots) for four unseen test poses, with per-pose RMSE.

geometry as the robot joints move. Examples of construction-robotics systems with substantial cable bundles are described in [5] and [6].

Previous work has addressed planning for deformable systems through component-level physical modeling and manipulation-based identification of material properties. For example, Gayle et al. [7] represented deformable systems using spring-mass-damper models. Such approaches require detailed physical characterization or finite-element analysis to obtain an accurate system model. More recent work has considered deformable-object manipulation. Aksoy and Wen [8], for example, estimated parameters such as Young's modulus from force and torque measurements. That setting assumes that the deformable object can be directly manipulated and sensed, whereas the robot-mounted cable considered here is not independently actuated or instrumented. Yin et al. [9]

survey data-driven methods for deformable-object handling; many of these methods similarly rely on direct interaction with or sensing of the deformable object.

We address this planning problem by training a neural network to predict quasi-static cable geometry as a function of robot joint angles, as illustrated in Fig. 1. We represent the cable using a pseudospectral Chebyshev interpolant with constrained endpoints. The resulting model supports efficient inference and provides the derivatives required for gradient-based trajectory optimization. In this work, we integrate the model with the Gaussian Process Motion Planner GPMP2 [10]. The representation may also be compatible with collision-avoidance formulations based on control barrier functions [11] or geometric potential fields [12], although these integrations are not evaluated here.

II. METHODOLOGY

We frame our planning problem as trajectory optimization, where the objective is to compute trajectories that satisfy constraints and minimize the cost as follows:

$$\begin{aligned} \min \quad & F[\theta(t)] \\ \text{subject to} \quad & \mathcal{G}_i[\theta(t)] \leq 0, \quad i = 1, \dots, m_{ineq}, \\ & \mathcal{H}_i[\theta(t)] = 0, \quad i = 1, \dots, m_{eq}, \end{aligned} \quad (1)$$

where $\theta(t) : t \rightarrow \mathbb{R}^D$. Here, D refers to the state dimensionality, and the trajectory $\theta(t)$ is a continuous-time function. The functionals $\mathcal{G}_i$ collect inequality constraints, such as joint limits and obstacle-clearance requirements, while $\mathcal{H}_i$ collect equality constraints, such as fixed start and goal states.

To model the cable geometry itself, we use a pseudospectral Chebyshev spline. Rather than parameterizing a polynomial by its coefficients, the pseudospectral approach parameterizes it by its values at a set of Chebyshev points of the second kind. Mapped onto the unit interval $s \in [0, 1]$, these points are given by

$$s_j = \frac{1}{2} \left(1 - \cos \frac{j\pi}{N-1} \right), \quad j = 0, \dots, N-1, \quad (2)$$

which cluster near the endpoints of the interval, avoiding the oscillations that arise when interpolating at equally spaced nodes. Given nodal values $\mathbf{p}_j = \mathbf{p}(s_j)$, the spline can then be evaluated at any $s \in [0, 1]$ via barycentric Lagrange interpolation:

$$\mathbf{p}(s) = \sum_{j=0}^{N-1} w_j(s) \mathbf{p}_j, \quad w_j(s) = \frac{\lambda_j / (s - s_j)}{\sum_{k=0}^{N-1} \lambda_k / (s - s_k)}, \quad (3)$$

where $\lambda_j = (-1)^j$, halved at the two endpoint nodes $j = 0$ and $j = N-1$. The weights satisfy $w_j(s_k) = 1$ if $j = k$ and 0 otherwise, so the interpolant passes exactly through its nodal values. Because evaluation is linear in the nodal values, the derivatives required for gradient-based trajectory optimization are readily available. If the reader has more questions about this approach, we encourage them to read [13].

We use this representation as the output parameterization of the neural network. The two cable attachment points are assumed to be known because each is fixed in the local frame of its corresponding robot link and can therefore be transformed into the selected reference frame. For a configuration θ , let $\mathbf{p}_A(\theta)$ and $\mathbf{p}_B(\theta)$ denote the attachment points anchored at the endpoint nodes $s_0 = 0$ and $s_{N-1} = 1$. Let $R(\theta)$ denote the rotation from the prediction frame to the selected reference frame, and let $\boldsymbol{\rho}(\theta) = g_\phi(S(\theta))$ denote the learned multi-layer perceptron with parameters ϕ . The selection operator S identifies the joint variables supplied to the network. The network predicts the $N-2$ interior Chebyshev nodal residuals $\boldsymbol{\rho} = [\boldsymbol{\rho}_1, \dots, \boldsymbol{\rho}_{N-2}]$, where $\boldsymbol{\rho}_j \in \mathbb{R}^3$, relative to the chord between the attachment points. The predicted cable position $\mathbf{x}^{cab}$ is

$$\begin{aligned} \mathbf{x}^{cab}(s; \theta) = & (1-s) \mathbf{p}_A(\theta) + s \mathbf{p}_B(\theta) \\ & + R(\theta) \sum_{j=1}^{N-2} w_j(s) \boldsymbol{\rho}_j(\theta), \quad s \in [0, 1], \end{aligned} \quad (4)$$

where $w_j(s)$ are the barycentric weights of Eq. (3). Since the residual vanishes at the attachment points, the sum runs over the interior nodes only.

III. EXPERIMENTAL SETUP

This section describes the integration of the learned cable model into a factor-graph-based planning framework, followed by the simulation platform, the evaluation task, and the network training procedure. We implement the motion-planning formulation using GTSAM [14] and GTDynamics [15].

A. Factor-Graph Planning Formulation

Following GPMP2 [10], motion planning is formulated as posterior estimation with a Gaussian-process prior. The problem is represented by a factor graph $G = \{\Theta, \mathcal{F}, \mathcal{E}\}$, where $\Theta = \{\theta_0, \dots, \theta_N\}$ is the set of variable nodes, $\mathcal{F} = \{f_0, \dots, f_M\}$ is the set of factor nodes, and $\mathcal{E}$ is the set of edges connecting variable and factor nodes. The posterior distribution factorizes as

$$p(\theta|e) \propto \prod_{m=1}^M f_m(\Theta_m). \quad (5)$$

The obstacle factor defined in [10] represents collision cost in the factor-graph formulation. The unary obstacle factor for a single configuration is

$$f_i^{obs}(\theta_i) = \exp \left\{ -\frac{1}{2} \|\mathbf{h}(\theta_i)\|_{\sigma_{obs}}^2 \right\}. \quad (6)$$

This factor penalizes collisions at trajectory support states but does not evaluate the interpolated trajectory between adjacent support states. We therefore also use the Gaussian-process interpolated obstacle factor:

$$\begin{aligned} f_{\tau_j}^{intp}(\theta_i, \theta_{i+1}) = & \exp \left\{ -\frac{1}{2} \|\mathbf{h}(\theta(\tau_j))\|_{\sigma_{obs}}^2 \right\} \\ = & \exp \left\{ -\frac{1}{2} \|\mathbf{h}_{\tau_j}^{intp}(\theta_i, \theta_{i+1})\|_{\sigma_{obs}}^2 \right\}. \end{aligned} \quad (7)$$

GPMP2 uses sphere-based collision primitives to evaluate these obstacle factors [10].

B. Cable Spline Factor

We extend this formulation with a *cable spline factor* that incorporates the learned cable geometry. The factor evaluates the cable curve in Eq. (4) at a given configuration θ . The cable cost vector stacks the sphere-based hinge loss $\mathbf{h}(\cdot)$ with standoff inflated by the cable radius r_{cab} at M samples $\{s_m\}$ along the curve:

$$\mathbf{h}^{cab}(\theta) = [\mathbf{h}(\mathbf{x}^{cab}(s_1; \theta)), \dots, \mathbf{h}(\mathbf{x}^{cab}(s_M; \theta))]^\top. \quad (8)$$

The unary cable spline factor follows the same form as the standard obstacle factor:

$$f_i^{cab}(\theta_i) = \exp \left\{ -\frac{1}{2} \|\mathbf{h}^{cab}(\theta_i)\|_{\sigma_{cab}}^2 \right\}. \quad (9)$$

The corresponding Gaussian-process interpolated cable factor is

$$\begin{aligned} f_{\tau_j}^{cab-intp}(\theta_i, \theta_{i+1}) &= \exp \left\{ -\frac{1}{2} \|\mathbf{h}^{cab}(\theta(\tau_j))\|_{\sigma_{cab}}^2 \right\} \\ &= \exp \left\{ -\frac{1}{2} \|\mathbf{h}^{cab-intp}(\theta_i, \theta_{i+1})\|_{\sigma_{cab}}^2 \right\}. \end{aligned} \quad (10)$$

C. Simulation Platform and Task

The simulated platform consists of a modified 6-DoF ABB IRB-6620 manipulator mounted on a 3-DoF gantry, as shown in Fig. 2. The cable is attached to links 3 and 6. Motion of these links changes the cable geometry and slack as a function of robot configuration and gravity. We evaluate

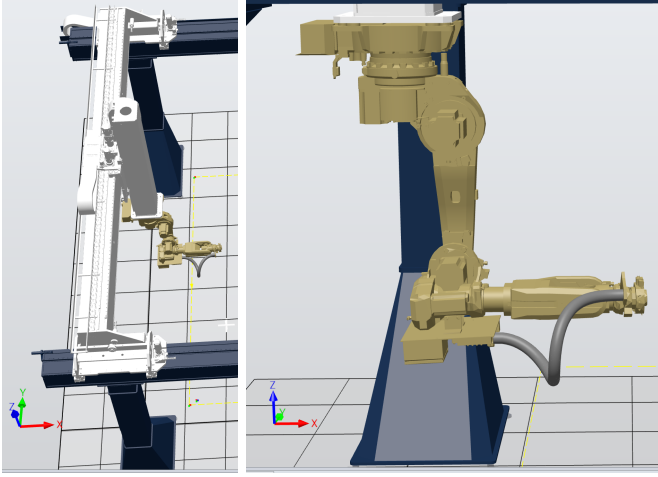


Fig. 2. Robot platform used in simulation. Left: Full 9-DoF robot and gantry assembly. Right: Cable attachment points on links 3 and 6.

the method by planning a trajectory through a window-like obstacle containing hooks, whose dimensions are shown in Fig. 3. The hooks create potential cable-snagging hazards that are not captured by a rigid-link collision model. The planning objective is to move the end effector through the opening to a target on the opposite side. The start and goal configurations

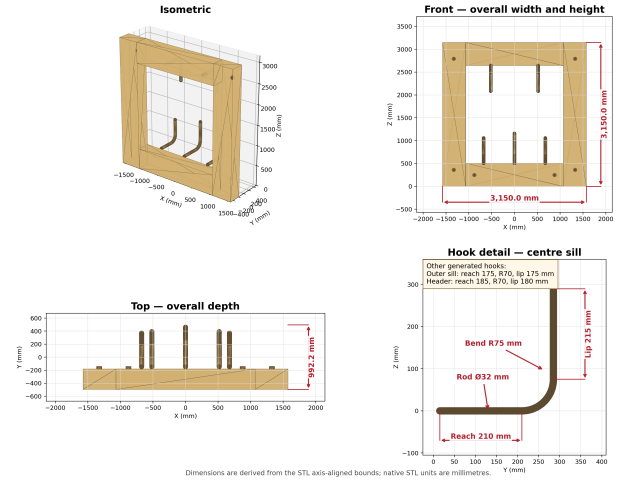


Fig. 3. Dimensions of the window-like obstacle and hooks used in the simulation experiments.

constrain joint 4 to 45 degrees, requiring the planner to account for the rotation of link 6 and the resulting change in cable geometry.

D. Network Training

To train the neural network, we generated 2,000 poses in RobotStudio using Sobol sampling, a quasi-random method that distributes samples across a multidimensional parameter space. Additional stationary configurations encountered during the simulated transitions were retained, yielding an augmented dataset of 5,718 samples. The network uses rectified linear unit (ReLU) activations between fully connected layers, and the dataset was divided into training, validation, and test subsets using a 70–15–15% split. The network was trained in the local frame of link 1; the rotation $R(\theta)$ in Eq. (4) therefore maps the predicted residuals from the link 1 frame to the selected reference frame.

IV. RESULTS

A. Collision-Free Planning

We compared the proposed model with two baselines: a planner that omitted the cable from collision checking and a planner that represented the cable as a straight segment between links 3 and 6. In the evaluated simulation scenario, only the proposed method produced a trajectory without a detected cable collision. Figure 4 shows the minimum-clearance or collision configurations for the three methods. Figure 5 shows the trajectory computed using the learned cable model and the corresponding predicted cable geometry. The optimized robot trajectory changes the cable configuration so that the predicted cable passes above the hook.

B. Ablation Studies

The architecture study indicates that a neural network with three hidden layers of width 64 achieves the lowest test RMSE (14.09 mm), with an inference time of approximately 100 μ s per sample, as shown in Table I. Figure 1 qualitatively

TABLE I
NEURAL-NETWORK ARCHITECTURE ABLATION ON THE FULL 5,718-SAMPLE DATASET. RMSE VALUES ARE IN MILLIMETERS; INFERENCE TIME IS PER SAMPLE ON CPU. THE ARCHITECTURE WITH THE LOWEST TEST RMSE IS SHOWN IN BOLD.

Hidden Layers	Width	Parameters	Train RMSE (mm)	Test RMSE (mm)	Inference (μ s)
1	8	426	57.99	58.41	43.15
1	16	810	42.29	45.40	44.62
1	32	1,578	28.94	35.99	46.11
1	64	3,114	16.83	24.66	44.17
1	128	6,186	9.99	22.83	40.02
2	8	498	41.10	42.37	76.52
2	16	1,082	23.26	29.91	65.38
2	32	2,634	10.69	19.09	62.15
2	64	7,274	6.26	17.22	60.57
2	128	22,698	4.05	15.32	78.70
3	8	570	30.76	35.97	81.66
3	16	1,354	16.06	25.05	81.16
3	32	3,690	8.57	18.79	88.20
3	64	11,434	4.33	14.09	101.90
3	128	39,210	4.07	15.05	127.24

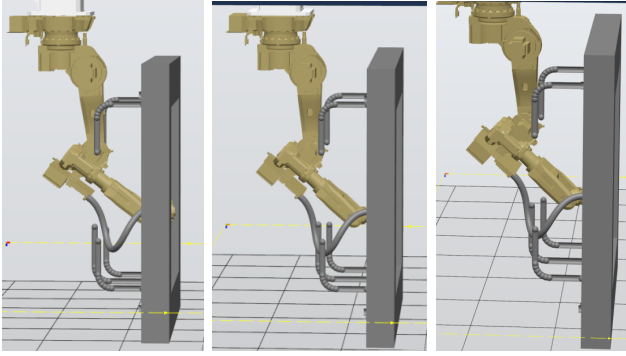


Fig. 4. Cable clearance for the three planning methods. Left: Proposed learned cable model. Center: Planner without cable prediction. Right: Straight-segment cable approximation.

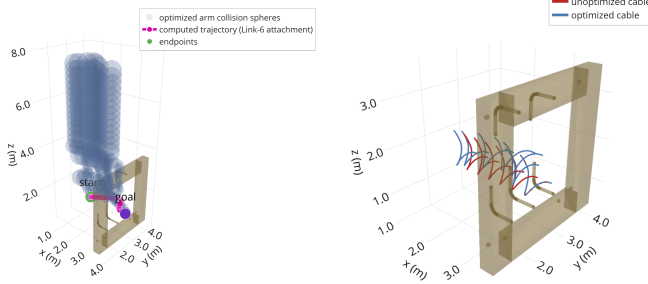


Fig. 5. Trajectory produced using the learned cable model. Left: Robot trajectory between the start and goal configurations. Right: Predicted cable geometry along the optimized trajectory.

compares the predicted and simulated cable geometries for four randomly selected test configurations. These examples supplement the aggregate RMSE by showing how prediction errors are distributed along the cable.

V. LIMITATIONS

Our chebyshev polynomial approach is limited by its ability to model local perturbations and lack of physical constraint modeling. One nodal residual in the model influences the global curve, which means it cannot represent localized kinks or discontinuities without raising the number of nodes. Similarly, our model does not have physical constraints built in, which means there is no collision avoidance on the arms built in, and rather relies on the training data supplying such cases so the model can learn them.

Another limitation of our proposed approach is the challenge of obtaining hardware validation. We compared it qualitatively, ensuring that the cable shape in simulation was indicative of reality. For greater precision and validation to real hardware, one must run a set of experiments using calibrated cameras and an image segmentation model.

VI. CONCLUSION

We presented a neural-network method for representing deformable cable geometry with an endpoint-constrained Chebyshev interpolant and incorporating the representation into factor-graph-based motion planning. In the evaluated simulation scenario, the learned model enabled the planner to avoid a cable collision that occurred when the cable was omitted or approximated as a straight segment. Future work will evaluate the method on coordinated dual-arm construction tasks represented within a shared factor graph and investigate its integration with geometric manipulation methods such as [16]. Incorporating LiDAR-based obstacle detection would also relax the present assumption that the obstacle geometry is known in advance.

ACKNOWLEDGMENT

K.S. thanks John D'Amato and Chisung Kim for their assistance with the simulation environment.

REFERENCES

- [1] Y. Liu, A. H. Alias, N. A. Haron, N. A. Bakar, and H. Wang, "Robotics in the construction sector: Trends, advances, and challenges," *J. Intell. Robot. Syst.*, vol. 110, no. 2, p. 72, May 2024.
- [2] G. Li, S. Liu, L. Wang, and R. Zhu, "Skin-inspired quadruple tactile sensors integrated on a robot hand enable object recognition," *Sci. Robot.*, vol. 5, no. 49, p. eabc8134, 2020.
- [3] P. Gsellmann, D. Natter, and G. Schitter, "Collision avoidance for a SCARA robot on construction sites," in *2022 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, Sapporo, Japan, 2022, pp. 508–513.
- [4] N. Kayhani, H. Taghaddos, and S. Behzadipour, "Construction equipment collision-free path planning using robotic approach," in *Proc. 35th Int. Symp. Autom. Robot. Constr. (ISARC)*, Berlin, Germany, 2018, pp. 1220–1225.
- [5] E. P. G. Bruun, S. Adriaenssens, and S. Parascho, "Structural rigidity theory applied to the scaffold-free (dis)assembly of space frames using cooperative robotics," *Automation in Construction*, vol. 141, p. 104405, 2022.
- [6] E. P. G. Bruun, R. Pastrana, V. Paris, A. Beghini, A. Pizzigoni, S. Parascho, and S. Adriaenssens, "Three cooperative robotic fabrication methods for the scaffold-free construction of a masonry arch," *Automation in Construction*, vol. 129, p. 103803, 2021.
- [7] R. Gayle, P. Segars, M. C. Lin, and D. Manocha, "Path planning for deformable robots in complex environments," in *Proceedings of Robotics: Science and Systems (RSS)*, Cambridge, MA, USA, 2005.
- [8] B. Aksoy and J. T. Wen, "Planning and control for deformable linear object manipulation," *IEEE Transactions on Automation Science and Engineering*, vol. 23, pp. 1093–1111, 2026.
- [9] H. Yin, A. Varava, and D. Kragic, "Modeling, learning, perception, and control methods for deformable object manipulation," *Sci. Robot.*, vol. 6, no. 54, p. eabd8803, 2021.
- [10] M. Mukadam, J. Dong, X. Yan, F. Dellaert, and B. Boots, "Continuous-time Gaussian process motion planning via probabilistic inference," *Int. J. Robot. Res.*, vol. 37, no. 11, pp. 1319–1340, Sep. 2018.
- [11] R. I. C. Muchacho, R. Laha, F. T. Pokorny, L. F. C. Figueredo, and N. Chakraborty, "Unifying complementarity constraints and control barrier functions for safe whole-body robot control," *IEEE Transactions on Automatic Control*, 2026.
- [12] Y. Gong, R. Laha, and L. Figueredo, "GeoPF: Infusing geometry into potential fields for reactive planning in non-trivial environments," *IEEE Robotics and Automation Letters*, 2025.
- [13] J.-P. Berrut and L. N. Trefethen, "Barycentric lagrange interpolation," *SIAM Review*, vol. 46, no. 3, pp. 501–517, 2004. [Online]. Available: <https://doi.org/10.1137/S0036144502417715>
- [14] F. Dellaert, "Factor graphs and GTSAM: A hands-on introduction," Georgia Institute of Technology, Tech. Rep. GT-RIM-CP&R-2012-002, 2012.
- [15] M. Xie, A. Escontrela, and F. Dellaert, "A factor-graph approach for optimization problems with dynamics constraints," *arXiv preprint arXiv:2011.06194*, 2020.
- [16] T. Löw, M. de Paula Assis Fonseca, V. Pruks, G. Deacon, J. Konstantinova, and S. Calinon, "Dual-arm admittance control using conformal geometric algebra," *Frontiers in Robotics and AI*, vol. 13, p. 1807613, 2026.